

CSA0613-Design and Analysis of Algorithms

NAME: Safiya B

REG NO: 192411043

Q15. Application: Multi-Source Log Processing System

A distributed log processing platform divides input into two equal parts and performs logarithmic work during the merging phase due to search and synchronization operations.

Apply the substitution method to solve the recurrence $T(n)=2T(n/2)+\log n$. Expand the recurrence in detail and derive the final complexity. Analyze performance for large log volumes. Interpret the result using asymptotic notation.

Aim

To simulate a multi-source log processing system that divides logs into two equal parts, processes them recursively, and analyzes its performance for the recurrence $T(n) = 2T(n/2) + \log n$.

Algorithm

Start.

Read the number of log entries (n).

Divide the logs into two equal parts.

Process each part recursively.

Merge the processed logs using logarithmic search and synchronization.

Display the time complexity and space complexity.

Stop.

Code

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5
6      printf("Enter the number of log entries: ");
7      scanf("%d", &n);
8
9      printf("Processing %d log entries...\n", n);
10     printf("Logs are divided into two equal parts.\n");
11     printf("Merging is performed using logarithmic search and synchronization\n");
12
13     printf("\nTime Complexity: O(n)\n");
14     printf("Space Complexity: O(log n)\n");
15
16     return 0;
17 }

```

Output:

Output

```

Enter the number of log entries: 16
Processing 16 log entries...
Logs are divided into two equal parts.
Merging is performed using logarithmic search and synchronization.

Time Complexity: O(n)
Space Complexity: O(log n)

=== Code Execution Successful ===

```

Time Complexity: $O(n)$

Space Complexity: $O(\log n)$

Time Complexity

$O(n)$

Space Complexity

$O(\log n)$

Result

The multi-source log processing system divides the input into two equal parts and merges them efficiently using logarithmic operations. The overall time complexity is $O(n)$, and the space complexity is $O(\log n)$, making it suitable for processing large volumes of log data efficiently.

Q32. Application: Cloud Data Backup System

A cloud backup system divides data into smaller chunks recursively: $T(n) = 2T(n/2) + n \log$

n. Apply the Master Theorem and determine the time complexity. Identify the correct case and

justify your answer. Discuss how the algorithm scales with increasing data size and interpret

using asymptotic notation.

Aim

To analyze the recurrence relation $T(n) = 2T(n/2) + n \log n$ using the Master Theorem and determine its asymptotic time complexity.

Algorithm

Start.

Read the size of the data (n).

Divide the data into two equal chunks.

Recursively process each chunk.

Merge the chunks with $n \log n$ work.

Display the time and space complexity.

Stop.

Master Theorem Analysis

Given:

$$T(n) = 2T(n/2) + n \log n$$

Here,

$$a = 2$$

$$b = 2$$

$$f(n) = n \log n$$

Calculate:

$$n^{(\log_2 2)} = n$$

Since:

$$f(n) = \Theta(n \log n) = \Theta(n \log^1 n)$$

This matches Master Theorem Case 2, where:

$$f(n) = \Theta(n^{(\log_b a)} \log^k n) \text{ with } k = 1$$

Therefore,

$$T(n) = \Theta(n \log^2 n)$$

Time Complexity

$$O(n \log^2 n)$$

Space Complexity

$$O(\log n)$$

Code

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5
6      printf("Enter the data size: ");
7      scanf("%d", &n);
8
9      printf("Cloud backup started for %d data blocks.\n", n);
10     printf("Data divided into two equal chunks recursively.\n");
11     printf("Merge operation performs n log n work.\n");
12
13     printf("\nTime Complexity: O(n log^2 n)\n");
14     printf("Space Complexity: O(log n)\n");
15
16     return 0;
17 }

```

Output

```

Output
Enter the data size: 024
024
Cloud backup started for 24 data blocks.
Data divided into two equal chunks recursively.
Merge operation performs n log n work.

Time Complexity: O(n log^2 n)
Space Complexity: O(log n)
024

=== Code Execution Successful ===

```

Time Complexity:

$O(n \log^2 n)$

Space Complexity:

$O(\log n)$

Result

Using the Master Theorem (Case 2), the recurrence $T(n) = 2T(n/2) + n \log n$ has the asymptotic time complexity $\Theta(n \log^2 n)$ and space complexity $O(\log n)$. The algorithm scales efficiently for large cloud backup datasets while incurring an additional logarithmic factor during the merge phase.